Lesson 5 Worksheet: Implementing the Manhattan Class — ANSWER KEY

Part 1: Code Tracing — compute_path() Step by Step

Trace the call: `Manhattan((0, 0)).compute_path((2, 3))`

Setup variables:

Variable	Initial Value
`self.position`	(0, 0)
`current_row`	0
`current_col`	0
`dest_row`	2
`dest_col`	3
`row_step`	1
`col_step`	1
`path` (initial)	[(0, 0)]

Row while loop — trace each iteration:

Iteration	current_row (before)	!= dest_row?	current_row (after)	Appended to path
1	0	YES	1	(1, 0)
2	1	YES	2	(2, 0)
Check	2	NO	— (loop ends)	—

Column while loop — trace each iteration:

Iteration	current_col (before)	!= dest_col?	current_col (after)	Appended to path
1	0	YES	1	(2, 1)
2	1	YES	2	(2, 2)
3	2	YES	3	(2, 3)
Check	3	NO	— (loop ends)	—

Final returned path: [(0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3)]

Part 2: Trace a Reverse Path

Trace the call: `Manhattan((2, 3)).compute_path((0, 1))`

Setup variables:

Variable	Initial Value
`current_row`	2
`current_col`	3
`dest_row`	0
`dest_col`	1
`row_step`	-1
`col_step`	-1

Row while loop:

Iteration	current_row (before)	current_row (after)	Appended
1	2	1	(1, 3)
2	1	0	(0, 3)

Column while loop:

Iteration	current_col (before)	current_col (after)	Appended
1	3	2	(0, 2)
2	2	1	(0, 1)

Final returned path: [(2, 3), (1, 3), (0, 3), (0, 2), (0, 1)]

Part 3: Fill in the Blanks

```
class Manhattan:
    def __init__(self, start):
        self.position = start

    def compute_path(self, destination):
        path = [self.position]

        current_row = self.position[0]
        current_col = self.position[1]
        dest_row = destination[0]
        dest_col = destination[1]

        if dest_row > current_row:
            row_step = 1
        else:
            row_step = -1

        if dest_col > current_col:
            col_step = 1
        else:
            col_step = -1

        while current_row ≠ dest_row:
            current_row = current_row + row_step
            path.append((current_row,  current_col))

        while current_col ≠ dest_col:
            current_col = current_col + col_step
            path.append((current_row,  current_col))

        return path
```

Blanks in order:

1. `position`

2. `self.position`
3. `0`
4. `1`
5. `destination`
6. `destination`
7. `>`
8. `1`
9. `-1`
10. `>`
11. `1`
12. `-1`
13. `≠`
14. `row_step`
15. `append`
16. `current_row`
17. `current_col`
18. `≠`
19. `col_step`
20. `append`
21. `current_row`
22. `current_col`
23. `path`

Part 4: Predict the Output

Program A:

Line 1: [(0, 0), (0, 1), (0, 2), (0, 3)]

Line 2: Steps: 3

What kind of edge case is this? Same row – the row loop does not execute, so the path only has column movement.

Program B:

Line 1: [(2, 2)]

Line 2: Steps: 0

Why does the path have only one element? The start equals the destination, so neither while loop executes. The only element in the path is the starting position added during initialization (`path = [self.position]`).

Program C:

Line 1: Path 1: [(0, 0), (1, 0), (2, 0), (2, 1), (2, 2), (2, 3)]

Line 2: Path 2: [(0, 0), (1, 0), (1, 1)]

Do both paths start from the same position? YES

Why? `compute_path` always uses `self.position` as the starting point, which is (0, 0) and never changes. The method does not update `self.position` – it only modifies local variables (`current_row` and `current_col`). So every call to `compute_path` starts from the original position (0, 0).

Part 5: Find and Fix the Bug

Bug 1:

What is wrong? `path` starts as an empty list `[]` instead of `[self.position]`. The starting position is missing from the path, so the returned path only contains the intermediate steps and destination but not where the robot begins.

Fix: Change `path = []` to `path = [self.position]`

Bug 2:

What is wrong? The row while loop uses `<` instead of `≠`. This fails when the robot needs to move in the negative row direction (north), because `current_row` starts greater than `dest_row`, so the condition `current_row < dest_row` is immediately False and the loop never runs.

When would this bug cause a problem? Give a specific start and destination:

Start: (3, 0) Destination: (0, 0)

Fix: Change `while current_row < dest_row:` to `while current_row ≠ dest_row:`

Bug 3:

What is wrong? In the column while loop, the append call has row and column swapped: `path.append((current_col, current_row))` instead of `path.append((current_row, current_col))`. This puts the column value in the row position and vice versa.

If start is (0, 0) and destination is (2, 3), what would the INCORRECT last three elements of the path be?

(1, 2), (2, 2), (3, 2)

What should they be?

(2, 1), (2, 2), (2, 3)

Fix: Change `path.append((current_col, current_row))` to `path.append((current_row, current_col))`

Reflection

The compute_path() method uses while loops instead of for loops. Why is `while current_row ≠ dest_row` better than using a for loop for this problem?

While loops are better because the number of iterations depends on the distance between start and destination, which varies with every call. We need to loop UNTIL we arrive rather than FOR a fixed count. The while condition `current_row ≠ dest_row` naturally handles both positive and negative directions – whether the robot moves north or south, the loop keeps going until the current row matches the destination row. A for loop would require calculating the distance first and handling direction separately, making the code more complicated.